CS 410/510 · DATABASES · FALL 2026 · INSTRUCTOR COPY

Normalization Practice — Answer Key

Worked solutions for the walkthrough and all five practice exercises

Monday, November 2 · Elmasri & Navathe, Chapter 14

Teaching notes are marked in brown throughout — they flag the specific wrong answer students reach for, so you can call it out before it spreads. The green blocks are the answers themselves.

Exercises 4 and 5 are a chain: 5 picks up where 4 ends. Exercise 5 restates its relation in full, so a student who botched 4 can still do 5.

WALKTHROUGH

Exercise 0 — Student advising records

Worked together at the start of class · target 1NF · about 12 minutes

```
STUDENT_ADVISING(StudentID, StudentName, Address, Phones, Courses)
```

TASK 1 — WHY THIS FAILS 1NF

ANSWER

Three distinct failures, and the distinction matters because each has a different fix:

1. **Address is composite.** Street, city, state, and zip are packed into one string. Nothing stops you storing it that way, but you cannot ask for every student in Boise without parsing text.
2. **Phones is multivalued.** A set lives in a single cell. The number of values varies by row.
3. **Courses is a nested relation.** Worse than multivalued — each element is itself a tuple with its own structure (course, term, grade).

1NF requires every attribute value to be atomic and single-valued. Only the third failure is what most students picture when they hear "not in 1NF."

TASK 2 — THE THREE STANDARD REMEDIES

ANSWER

R1 — Expand the key. Flatten by repeating the other attributes, making the key `{StudentID, Phone, CourseNum, Term}`.

Cost: Marisol has 2 phones and 2 courses, so she becomes 4 rows — a spurious cross product. Phones and courses are independent facts about a student, and this remedy invents a pairing between them that does not exist. It also introduces heavy redundancy.

R2 — One relation per multivalued attribute. Move each multivalued or nested attribute into its own relation keyed by the original key plus whatever distinguishes the values.

Cost: more relations, and a join to reassemble the original view. This is the right answer and the one we will use.

R3 — Fixed columns. If the maximum number of values is known, replace the attribute with that many atomic columns: `Phone1, Phone2, Phone3`.

Cost: a hard cap, NULL padding, and queries that must search every column. This is a repeating group — see Exercise 2, where it is the whole point.

Watch for: students proposing R1 and liking it, because it produces a single flat table and "looks normalized." Draw the four rows on the board. The cross product is obvious once it is visible and almost invisible as prose. If anyone asks whether R2 fully solves the independence problem, that is the multivalued dependency question — it is on today's slides, and you can promise it rather than answer it here.

TASK 3 — THE 1NF SCHEMA

ANSWER

```
STUDENT(StudentID, StudentName, Street, City, State, Zip)
    PK  StudentID

STUDENT_PHONE(StudentID, Phone)
    PK  {StudentID, Phone}
    FK  StudentID → STUDENT

ENROLLMENT(StudentID, CourseNum, Term, Grade)
    PK  {StudentID, CourseNum, Term}
    FK  StudentID → STUDENT
```

Address was solved by splitting columns, not by a new relation — a composite attribute never needs one. The two genuinely multivalued facts each got their own relation, which is why there are three relations and not two.

`Term` is in the ENROLLMENT key because a student may retake a course. Drop it from the key and you cannot record a repeat.

1NF Exercise 1 — Senior design projects

About 8 minutes

TASK 1 — CLASSIFY THE VIOLATIONS

ANSWER

- `SponsorContact` — **composite.** Name, email, and phone in one string.
- `TeamMembers` — **multivalued.** A set of student IDs.
- `Milestones` — **nested relation.** Each element is a (name, date) tuple.

`Sponsor` is fine: one atomic value per project.

TASK 2 — 1NF DECOMPOSITION

ANSWER

```
PROJECT(ProjectID, ProjectTitle, Sponsor, ContactName,
ContactEmail, ContactPhone)
    PK  ProjectID

PROJECT_MEMBER(ProjectID, StudentID)
    PK  {ProjectID, StudentID}
    FK  ProjectID → PROJECT,  FK StudentID → STUDENT

PROJECT_MILESTONE(ProjectID, MilestoneName, DueDate)
    PK  {ProjectID, MilestoneName}
    FK  ProjectID → PROJECT
```

TASK 3 — WHY TWO RELATIONS, NOT ONE

ANSWER

Team membership and milestone schedule are independent facts about a project. Combining them into one relation forces a row for every (member, milestone) pair: three members and three milestones become nine rows, asserting a connection between Dana's prototype deadline and a particular student that the mini-world does not contain.

> The rule: each independent multivalued fact gets its own relation. This is exactly why remedy R1 was rejected in the walkthrough — worth drawing the parallel aloud.

Watch for: students splitting `SponsorContact` into its own relation. It is composite, not multivalued — one project has exactly one contact, so splitting the columns is sufficient. A separate CONTACT relation is defensible only if a contact serves many projects, which nothing here states.

1NF Exercise 2 — Advisor load

About 8 minutes · this one is a trap, deliberately

TASK 1 — IS IT IN 1NF?

ANSWER

Yes. Formally, this relation is in First Normal Form.

Every cell holds exactly one atomic value. `Advisee2` is a single student ID, not a set. The definition of 1NF is satisfied, and no amount of disliking the design changes that.

This is the lesson: **1NF is a weak test.** Passing it tells you almost nothing about whether the design is any good. That is precisely why 2NF, 3NF, and BCNF exist.

TASK 2 — FOUR SPECIFIC PROBLEMS

ANSWER

`Advisee1..3` form a **repeating group**:

1. **Hard cap.** A fourth advisee requires altering the table. The schema encodes a business rule that is not a business rule.
2. **NULL padding.** Nakamura advises one student and carries four NULLs. Every aggregate must work around them.
3. **Query pain.** "Who advises S1198?" becomes three equality tests joined by OR, or a three-branch UNION. Adding a fourth slot breaks every such query already written.
4. **Misplaced attribute.** `Major` is a fact about a student, not about an advising slot. Stored here, a student's major is recorded in whichever advisor row happens to hold them.

TASK 3 — THE FIX

ANSWER

```
ADVISOR(AdvisorID, AdvisorName)
    PK  AdvisorID

STUDENT(StudentID, Major, AdvisorID)
```

```
    PK  StudentID
    FK  AdvisorID → ADVISOR
```

No bridge table is needed because advising is **1:N**, not M:N — each student has one advisor. The foreign key goes on the many side. A bridge table would be correct only if a student could have several advisors simultaneously, and it would additionally permit that, which the mini-world does not.

Note the payoff: `Major` lands on STUDENT, where it belongs, as a side effect of fixing the repeating group.

Watch for: a confident "no, it's not in 1NF" from most of the room. Do not let it pass as close enough. The whole point is separating *violates a normal form* from *is a bad design*; students who blur the two will guess on the exam instead of applying the definition.

2NF Exercise 3 — Enrollment

About 10 minutes

F1 `StudentID → StudentName, StudentMajor`

F2 `CourseNum → CourseTitle, Credits`

F3 `{StudentID, CourseNum} → Term, Grade`

TASK 1 — CANDIDATE KEY

ANSWER

Candidate key: {StudentID, CourseNum}

```
{StudentID, CourseNum}+
  = {StudentID, CourseNum}
  + {StudentName, StudentMajor}      by F1
  + {CourseTitle, Credits}           by F2
  + {Term, Grade}                    by F3
  = all eight attributes  →  it is a superkey
```

Minimality — check each part alone:

```
{StudentID}+ = {StudentID, StudentName, StudentMajor}   — incomplete
{CourseNum}+ = {CourseNum, CourseTitle, Credits}     — incomplete
```

Neither proper subset is a superkey, so the pair is a candidate key, and it is the only one.

TASK 2 — 2NF CHECK

ANSWER

In 1NF, not in 2NF.

2NF requires every non-prime attribute to be fully functionally dependent on the whole key. Two partial dependencies break it:

- **F1** — StudentName and StudentMajor depend on `StudentID` alone, which is half the key.

- **F2** — CourseTitle and Credits depend on `CourseNum` alone.

F3 is a full dependency and is fine.

TASK 3 — ANOMALIES

ANSWER

- **Insert.** A new course, CS 455, is approved but nobody has registered. There is no row to put it in — the key needs a StudentID. The course cannot exist until a student enrolls.
- **Update.** CS 410 is retitled. Every enrolled student's row must change. Miss one and the database holds two titles for one course.
- **Delete.** The last student drops CS 321. Deleting that row erases the fact that CS 321 exists and is worth 3 credits.

All three are the same underlying fault: course facts have no home of their own.

TASK 4 — 2NF DECOMPOSITION

ANSWER

```
STUDENT(StudentID, StudentName, StudentMajor)     PK  StudentID
COURSE(CourseNum, CourseTitle, Credits)           PK  CourseNum
ENROLLMENT(StudentID, CourseNum, Term, Grade)     PK  {StudentID,
CourseNum}
                                                  FK  StudentID, FK
CourseNum
```

Each partial dependency became its own relation, keyed by the determinant. All three results happen to be in 3NF as well — worth saying aloud, since it previews Exercise 5 and shows that the normal forms are a ladder, not a checklist.

Lossless: the joins are on `StudentID` and `CourseNum`, each the key of the relation it points to. Dependency-preserving: F1 lives in STUDENT, F2 in COURSE, F3 in ENROLLMENT.

2NF Exercise 4 — Section roster

About 10 minutes · the distractors are the point

F1 `CourseNum → CourseTitle, Credits`

F2 `{CourseNum, SectionNum, Semester} → InstructorID, Room, SeatsTaken`

F3 `InstructorID → InstructorName`

F4 `Room → RoomCapacity`

TASK 1 — CANDIDATE KEY

ANSWER

Candidate key: {CourseNum, SectionNum, Semester}

```
{CourseNum, SectionNum, Semester}+
  = key
  + {CourseTitle, Credits}              by F1
  + {InstructorID, Room, SeatsTaken}    by F2
  + {InstructorName}                    by F3
  + {RoomCapacity}                      by F4
  = all ten attributes  →  superkey
```

Minimality — all three two-attribute subsets:

```
{CourseNum, SectionNum}+ = + CourseTitle, Credits    — incomplete
{CourseNum, Semester}+   = + CourseTitle, Credits    — incomplete
{SectionNum, Semester}+  = itself                    — incomplete
```

None reaches all attributes, so the three-attribute key is minimal and unique.

TASK 2 — WHICH DEPENDENCY IS PARTIAL

ANSWER

F1 is the only partial dependency.

`CourseNum` is a proper subset of the key, and CourseTitle and Credits are non-prime, so they depend on part of the key. That violates 2NF.

The other three:

- **F2** — determinant is the entire key. A full dependency by definition.
- **F3** — `InstructorID` is *not* part of the key. It is a non-prime attribute, so F3 is a **transitive** dependency, not a partial one.
- **F4** — same reasoning. `Room` is non-prime, so F4 is transitive.

2NF says nothing about transitive dependencies. They are 3NF's business.

Watch for: students flagging F3 and F4 as partial because InstructorName and RoomCapacity "don't depend on the whole key." They do — through an intermediate attribute. The test for *partial* is whether the determinant is a proper subset *of the key*. If the determinant is not part of the key at all, the dependency cannot be partial. This single confusion accounts for most lost points on normalization exam questions; it is worth two extra minutes here.

TASK 3 — 2NF DECOMPOSITION

ANSWER

```
COURSE(CourseNum, CourseTitle, Credits)
    PK  CourseNum

SECTION_ROSTER(CourseNum, SectionNum, Semester, InstructorID,
InstructorName,
               Room, RoomCapacity, SeatsTaken)
    PK  {CourseNum, SectionNum, Semester}
    FK  CourseNum → COURSE
```

Only the partial dependency was removed. Nothing else moves at this stage.

TASK 4 — WHAT STILL BREAKS

ANSWER

SECTION_ROSTER is in 2NF and still violates 3NF, because F3 and F4 remain as transitive dependencies:

```
{CourseNum, SectionNum, Semester} → InstructorID → InstructorName
{CourseNum, SectionNum, Semester} → Room         → RoomCapacity
```

COURSE is already in 3NF — its only dependency is from the key.

3NF Exercise 5 — Section roster, continued

About 12 minutes

F2 `{CourseNum, SectionNum, Semester} → InstructorID, Room, SeatsTaken`

F3 `InstructorID → InstructorName, InstructorDept`

F4 `Room → RoomCapacity`

TASK 1 — 2NF YES, 3NF NO

ANSWER

Key is `{CourseNum, SectionNum, Semester}` ; its closure reaches every attribute via F2, then F3, then F4.

2NF holds: no determinant here is a proper subset of the key. `InstructorID` and `Room` are non-prime.

3NF fails. Two transitive dependencies:

```
key → InstructorID → InstructorName, InstructorDept
key → Room         → RoomCapacity
```

3NF requires that no non-prime attribute be transitively dependent on a key — equivalently, that every determinant be a superkey or every dependent attribute be prime. Neither `InstructorID` nor `Room` is a superkey.

TASK 2 — ANOMALIES FROM THE ROOM DEPENDENCY

ANSWER

- **Insert.** CCP-250 is renovated to seat 80. There is nowhere to record that until some section is scheduled into it — room capacity has no independent home.
- **Delete.** The only section in CCP-243 is canceled. Deleting the row destroys the fact that CCP-243 seats 48, even though the room still exists.

The update anomaly rounds it out: re-measuring a room means editing every section scheduled there.

TASK 3 — 3NF DECOMPOSITION

ANSWER

```
INSTRUCTOR(InstructorID, InstructorName, InstructorDept)
    PK  InstructorID

ROOM(Room, RoomCapacity)
    PK  Room

SECTION(CourseNum, SectionNum, Semester, InstructorID, Room,
SeatsTaken)
    PK  {CourseNum, SectionNum, Semester}
    FK  InstructorID → INSTRUCTOR
    FK  Room → ROOM
```

Each transitively determined group moved to a relation keyed by its determinant; the determinant stays behind as a foreign key. That is the entire 3NF decomposition procedure.

TASK 4 — LOSSLESS JOIN

ANSWER

- SECTION joins INSTRUCTOR on `InstructorID`, which is the key of INSTRUCTOR.
- SECTION joins ROOM on `Room`, which is the key of ROOM.

The binary test: a decomposition of R into R1 and R2 is lossless if the common attributes form a superkey of at least one fragment. Here each common attribute is the key of the fragment it points into, so each join is lossless, and applying the test successively makes the whole decomposition lossless.

The intuition worth saying out loud: every section row still knows exactly which instructor and which room it had, so joining back cannot manufacture a pairing that was not there.

TASK 5 — DEPENDENCY PRESERVATION

ANSWER

`Every original dependency is enforceable on a single relation.`

Dependency	Enforced by
F2 `key → InstructorID, Room, SeatsTaken`	`SECTION`
F3 `InstructorID → InstructorName, InstructorDept`	`INSTRUCTOR`
F4 `Room → RoomCapacity`	`ROOM`

Nothing requires a join to check, so the decomposition is dependency-preserving as well as lossless. 3NF decomposition always guarantees both — which is exactly what BCNF cannot always promise, and why 3NF survives in practice.

Closing the loop: fold in COURSE from Exercise 4 and the full schema — COURSE, INSTRUCTOR, ROOM, SECTION — is in 3NF.
